

ERHVERVSAKADEMI
AARHUS

Programmering modul 03

CSIK E2026

ERHVERVSAKADEMI
AARHUS

I dag

Betingede udtryk igen

Lokale variabler

Selektion

Iteration

5 + 20 + 5 minutters pause undervejs

Slutter 16:00

ERHVERVSAKADEMI
AARHUS

Betingede udtryk / conditionals

Betingede udtryk / conditionals

```
code(e1 then e2 else e3)  = code(e1)
                            jmpf else
                            code(e2)
                            jump done
                            else:
                            code(e3)
                            done:
```

Betingede udtryk / conditionals

```
code(e1 then e2 else e3)  = code(e1)
                            jmpf else
                            code(e2)
                            jump done
                            else:
                            code(e3)
                            done:

code(e1 and e2)           = code(e1 then e2 else false)
                          = code(e1)
                            jmpf else
                            code(e2)
                            jump done
                            else:
                            push false
                            done:
```

Betingede udtryk / conditionals

```
code(e1 then e2 else e3)  = code(e1)
                            jmpf else
                            code(e2)
                            jump done
                            else:
                            code(e3)
                            done:

code(e1 or e2)            = code(e1 then true else e2)
                          = code(e1)
                            jmpf else
                            push true
                            jump done
                            else:
                            code(e2)
                            done:
```

Betingede udtryk / conditionals

```
code(e1 then e2 else e3)  = code(e1)
                            jmpf else
                            code(e2)
                            jump done
                            else:
                            code(e3)
                            done:

code(a < b then a else b) =
```

Betingede udtryk / conditionals

```
code(e1 then e2 else e3)  = code(e1)
                            jmpf else
                            code(e2)
                            jump done
                            else:
                            code(e3)
                            done:

code(a < b then a else b) = code(a < b)


                            jmpf else
                            code(a)
                            jump done
                            else:
                            code(b)
                            done:
```

Betingede udtryk / conditionals

```
code(e1 then e2 else e3)  = code(e1)
                            jmpf else
                            code(e2)
                            jump done
                            else:
                            code(e3)
                            done:

code(a < b then a else b) = code(a)
                            code(b)
                            lt
                            jmpf else
                            code(a)
                            jump done
                            else:
                            code(b)
                            done:
```

Betingede udtryk / conditionals

```
code(e1 then e2 else e3)  = code(e1)
                            jmpf else
                            code(e2)
                            jump done
                            else:
                            code(e3)
                            done:

code(a < b then a else b) = load 0
                            load 1
                            lt
                            jmpf else
                            load 0
                            jump done
                            else:
                            load 1
                            done:
```

Betingede udtryk / conditionals

```
code(e1 then e2 else e3)  = code(e1)
                            jmpf else
                            code(e2)
                            jump done
                            else:
                            code(e3)
                            done:

code(a < b then a else b) = load 0
                            load 1
                            lt
                            jmpf else
                            load 0
                            jump done
                            else:
                            load 1
                            done:
```

```
let a = read in
let b = read in
  write (a < b then a else b)
end let
```

Betingede udtryk / conditionals

```
code(e1 then e2 else e3)  = code(e1)
                            jmpf else
                            code(e2)
                            jump done
                            else:
                            code(e3)
                            done:

code(a < b then a else b) = load 0
                            load 1
                            lt
                            jmpf else
                            load 0
                            jump done
                            else:
                            load 1
                            done:
```

```
let a = read in
let b = read in
  write (a < b then a else b)
end let
```

```
let a = read in
let b = read in
  if a < b then
    write a
  else
    write b
  end if
end let
```

ERHVERVSAKADEMI
AARHUS

Codelabs

Genbesøg

- "Imperative programming" side 33-36
- "Stack machine" side 35-48

Sørg for, at hele teamet er med. Hjælp hinanden.

Tag jer god tid, så I får læring ud af det

Pause 13:15–13:20

ERHVERVSAKADEMI
AARHUS



ERHVERVSAKADEMI
AARHUS

Lokale variabler

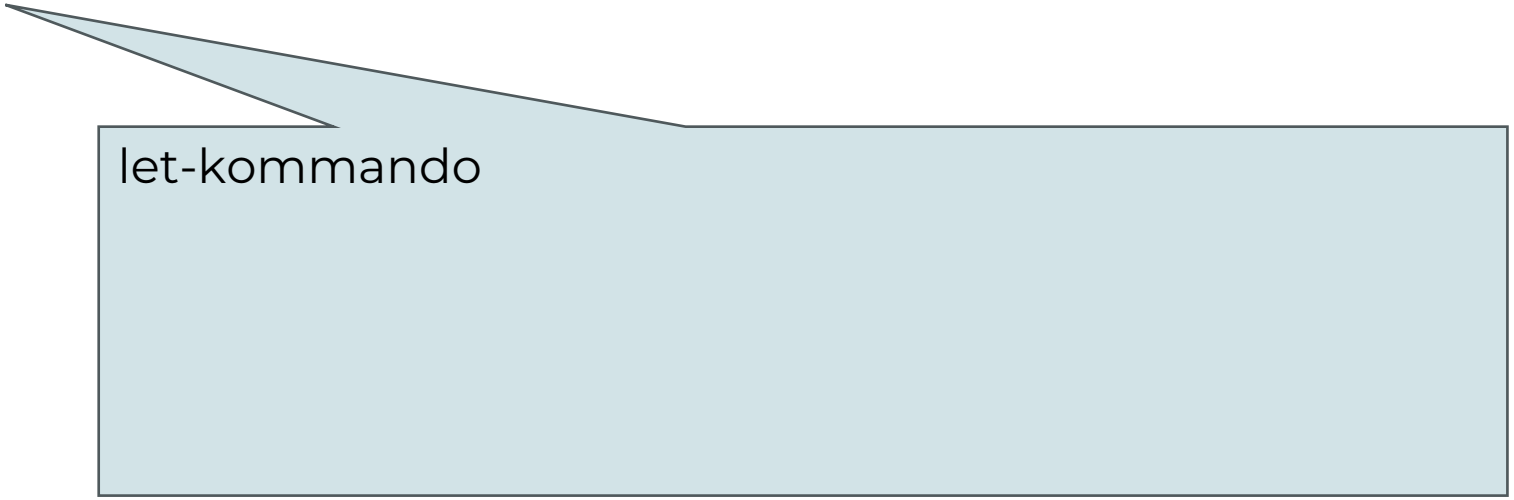
Lokale variabler

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

Lokale variabler: syntaks

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
let v = e in
  s
end let
```



Lokale variabler: syntaks

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
let v = e in
  s
end let
```

v er en variabel

= navn på lagerplads

Lokale variabler: syntaks

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
let v = e in
  s
end let
```

e er et udtryk / expression

Beskriver variablens initielle værdi

Lokale variabler: syntaks

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
let v = e in
  s
end let
```

s er en kommando / statement

let-kommandoens **krop**
= **scope** af v
= den kode, hvor v eksisterer

Lokale variabler: syntaks

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
let v = e in
  s
end let
```

`s` kan selv være en let-kommando

dvs. let-kommandoer kan være **indlejrede** i hinanden

Lokale variabler: syntaks

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
let v1 = e1 in
  let v2 = e2 in
    let v3 = e3 in
      s
    end let
  end let
end let
```

Det bliver hurtigt noget bøvl at skrive...

Lokale variabler: syntaks

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
let v1 = e1 in
let v2 = e2 in
let v3 = e3 in
  s
end let
```

Kort form, med præcis samme betydning

ERHVERVSAKADEMI
AARHUS

Lokale variabler: semantik

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
let v = e in
  s
end let
```

1. Evaluér **e**
2. Gem værdien i lagerplads kaldet **v**
3. Afvikl **s**
4. Drop bindingen til **v** og frigiv lagerpladsen

ERHVERVSAKADEMI
AARHUS

Lokale variabler: semantik

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
code( let v = e in
        s
      end let ) = code(e)
                  code(s)
                  pop
```

1. Evaluér **e**
2. Gem værdien i lagerplads kaldet **v**
3. Afvikl **s**
4. Drop bindingen til **v** og frigiv lagerpladsen

I stakmaskinen er lagerpladsen første ledige celle på stakken

Lokale variabler: semantik

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
code( let v = e in
        s
      end let ) = code(e)
                  code(s)
                  pop
```

```
code(v)           = load n
```

– hvis der er n variabler på stakken under v

1. Evaluér e
2. Gem værdien i lagerplads kaldet v
3. Afvikl s
4. Drop bindingen til v og frigiv lagerpladsen

v er et gyldigt udtryk i s

Det evalueres ved at aflæse værdien det sted i lageret, vi kalder v

ERHVERVSAKADEMI
AARHUS

Lokale variabler: semantik

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
let a = read in
  write a + 7
end let
```


1. Evaluér **e**
2. Gem værdien i lagerplads kaldet **v**
3. Afvikl **s**
4. Drop bindingen til **v** og frigiv lagerpladsen

v er et gyldigt udtryk i **s**

Det evalueres ved at aflæse værdien det sted i lageret, vi kalder **v**

Lokale variabler: semantik

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
let a = read in
let b = read in
  write a + b
end let
```


1. Evaluér **e**
2. Gem værdien i lagerplads kaldet **v**
3. Afvikl **s**
4. Drop bindingen til **v** og frigiv lagerpladsen

v er et gyldigt udtryk i **s**

Det evalueres ved at aflæse værdien det sted i lageret, vi kalder **v**

Lokale variabler: semantik

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
code( let v = e in
        s
      end let )  = code(e)
                   code(s)
                   pop
```

```
code(v)                 = load n
code(set v = e)         = code(e)
                          save n
```

– hvis der er n variabler på stakken under v

1. Evaluér e
2. Gem værdien i lagerplads kaldet v
3. Afvikl s
4. Drop bindingen til v og frigiv lagerpladsen

Vi kan tilordne en ny værdi til v i s

Den nye værdi overskriver den gamle

ERHVERVSAKADEMI
AARHUS

Lokale variabler: semantik

Lokal variabel = midlertidig, navngivet lagerplads,
hvor vi kan gemme værdien af et udtryk

```
let a = read in
  write a
  set a = a + 2
  write a
end let
```


1. Evaluér **e**
2. Gem værdien i lagerplads kaldet **v**
3. Afvikl **s**
4. Drop bindingen til **v** og frigiv lagerpladsen

Vi kan tilordne en ny værdi til **v** i **s**

Den nye værdi overskriver den gamle

ERHVERVSAKADEMI
AARHUS

Codelabs

Følg slagplanen fra samarbejdsstrukturer03.pdf på Canvas

Tag jer god tid, så I får læring ud af det. I når ikke det hele nu.

Pause 14:05-14:25

ERHVERVSAKADEMI
AARHUS



ERHVERVSAKADEMI
AARHUS

Selektion

Selektion

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

Selektion

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

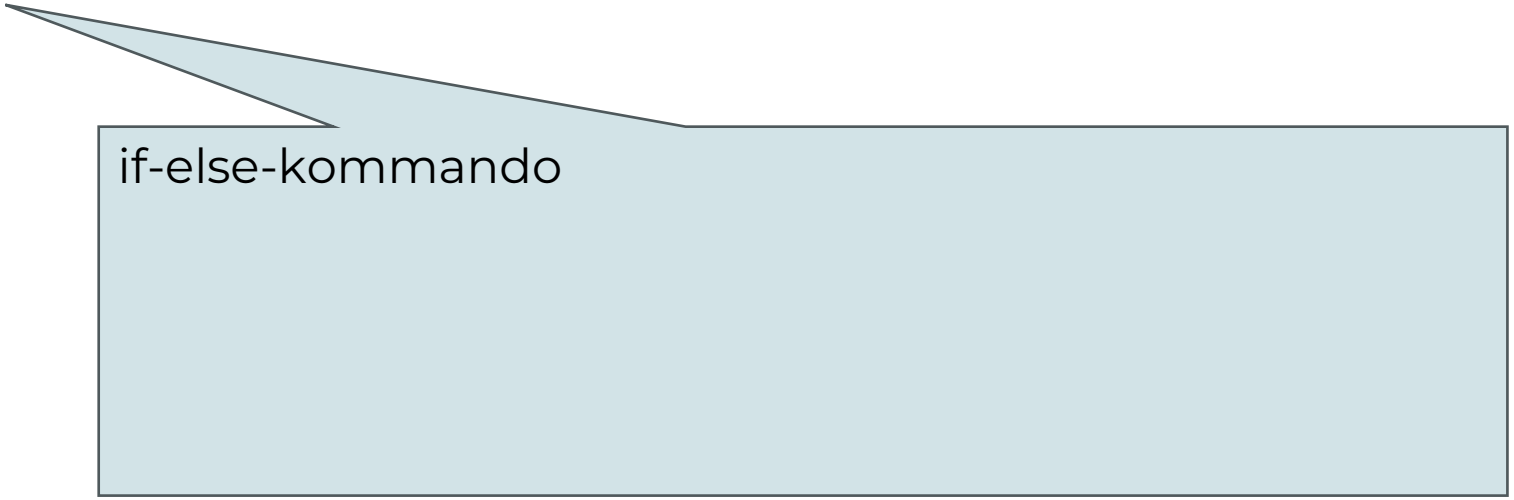
```
let a = read in
let b = read in
  write (a < b then a else b)
end let
```

```
let a = read in
let b = read in
  if a < b then
    write a
  else
    write b
  end if
end let
```

Selektion: syntaks

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

```
if e then
  s1
else
  s2
end if
```



Selektion: syntaks

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

```
if e then
   s1
else
   s2
end if
```

e er et udtryk

if-else-kommandoens **betingelse**

Selektion: syntaks

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

```
if e then
   s1
else
   s2
end if
```

`s1` er en kommando

if-else-kommandoens **then-gren**

Selektion: syntaks

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

```
if e then
   s1
else
   s2
end if
```

`s2` er en kommando

if-else-kommandoens **else-gren**

Selektion: syntaks

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

```
if e then
  s1
else
  s2
end if
```

`s1` og `s2` kan selv være if-else-kommandoer

dvs. if-else-kommandoer kan være indlejrede i hinanden

det sker fx ofte, at man har brug for at vælge mellem mere end to forskellige kommandoer

Selektion: syntaks

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

```
if e1 then
  s1
else
  if e2 then
    s2
  else
    if e3 then
      s3
    else
      s4
    end if
  end if
end if
```

Det bliver hurtigt noget bøvl at skrive...

Selektion: syntaks

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

```
if e1 then
  s1
else if e2 then
  s2
else if e3 then
  s3
else
  s4
end if
```

Kort form, med præcis samme betydning

Selektion: syntaks

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

```
if e then
  s1
else
  s2
end if
```

Det hænder også, at man har brug for at vælge mellem en kommando eller ingenting

Selektion: syntaks

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

```
if e then
  s
else
  skip
end if
```

Det er også lidt noget bøvl at skrive...

Selektion: syntaks

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

```
if e then
   s
end if
```

Kort form, med præcis samme betydning

Kaldes en if-kommando

Selektion: semantik

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

```
code( if e then
        s1
      else
        s2
      end if ) = code(e)
                 jmpf else
                 code(s1)
                 jump done
                 else:
                 code(s2)
                 done:
```

1. Evaluér `e`
2. Hvis resultatet er **`true`**, så afvikl `s1` ellers afvikl `s2`

Samme skabelon som ved betingede udtryk – nu bare med kommandoer i grenene i stedet for udtryk

ERHVERVSAKADEMI
AARHUS

Selektion: semantik

Selektion = at vælge mellem forskellige kommandoer baseret på værdien af et udtryk

```
code( if e then
        s1
      else
        s2
      end if ) = code(e)
                 jmpf else
                 code(s1)
                 jump done
                 else:
                 code(s2)
                 done:
```

```
code( if e then
        s
      end if ) = code(e)
                 jmpf done
                 code(s)
                 done:
```

1. Evaluér **e**
2. Hvis resultatet er **true**, så afvikl **s1**, ellers afvikl **s2**

if-kommandoen har en simplere skabelon:

1. Evaluér **e**
2. Hvis resultatet er **true**, så afvikl **s**, ellers fortsæt

ERHVERVSAKADEMI
AARHUS

Codelabs

Følg slagplanen fra samarbejdsstrukturer03.pdf på Canvas

Tag jer god tid, så I får læring ud af det. I når ikke det hele nu.

Pause 15:10-15:15

ERHVERVSAKADEMI
AARHUS



ERHVERVSAKADEMI
AARHUS

Iteration

Iteration

Iteration = at udføre den samme kommando gentagne gange, under kontrol af et udtryk

Iteration: syntaks

Iteration = at udføre den samme kommando gentagne gange, under kontrol af et udtryk

```
while e do
   s
end while
```

while-kommando = while-løkke

Iteration: syntaks

Iteration = at udføre den samme kommando gentagne gange, under kontrol af et udtryk

```
while e do
   s
end while
```

e er et udtryk

while-løkkens **betingelse**

Iteration: syntaks

Iteration = at udføre den samme kommando gentagne gange, under kontrol af et udtryk

```
while e do
  s
end while
```

`s` er en kommando

while-løkkens **krop**

Iteration: semantik

Iteration = at udføre den samme kommando gentagne gange, under kontrol af et udtryk

```
code( while e do
        s
      end while ) = loop:
                    code(e)
                    jmpf done
                    code(s)
                    jump loop
                    done:
```

1. Evaluér **e**
2. Hvis resultatet er **true**, så afvikl **s** og gentag fra step 1 ellers stop

Iteration: semantik

Iteration = at udføre den samme kommando gentagne gange, under kontrol af et udtryk

```
code( repeat s until e )    = loop:
                                code(s)
                                code(e)
                                jmpf loop
```

```
code( do s while e )        = loop:
                                code(s)
                                code(e)
                                jmpt loop
```

Nogle sprog har andre slags løkker, hvor man altid får afviklet kroppen mindst én gang

De kaldes fx repeat-until løkker og do-while-løkker.

Til oversættelsen af do-while-løkker, er det nyttigt at udvide stakmaskinen med en **`jmpt`**-instruktion

ERHVERVSAKADEMI
AARHUS

Codelabs

Følg slagplanen fra samarbejdsstrukturer03.pdf på Canvas

Tag jer god tid, så I får læring ud af det. I når ikke det hele nu.

Opsamling 15:50

ERHVERVSAKADEMI
AARHUS

Opsamling

Lokale variabler

Selektion

Iteration

Hvad var svært at forstå?

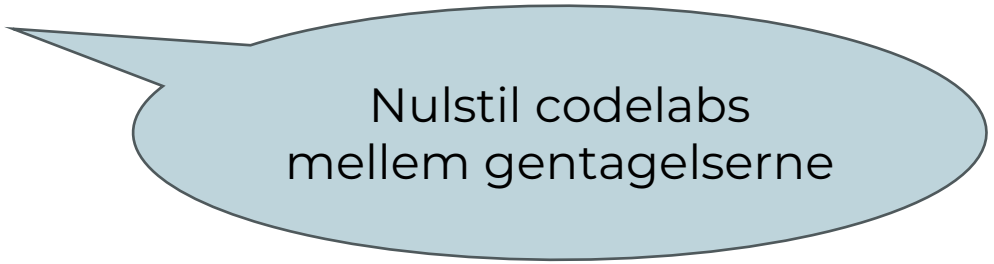
ERHVERVSAKADEMI
AARHUS

Opfølgende hjemmearbejde

Gennemgå "Imperative programming" side 42-74 flere gange

Gennemgå "Stack machine" side 49-62 flere gange

Det er øvelsesbaner, ikke afleveringer



Bliv ved, indtil

- teksten føles velkendt og let forståelig
- du kan løse opgaverne uden hjælp

Brug din studiegruppe så meget som muligt

- brug de nye begreber mundtligt: spørg, forklar, eksemplificér
- vær konkret og specifik

ERHVERVSAKADEMI
AARHUS

Opfølgende hjemmearbejde

Gennemgå "Imperative programming" side 42-74 flere gange

Gennemgå "Stack machine" side 49-62 flere gange

Det er øvelsesbaner, ikke afleveringer

Hjernen skal have
tid og gentagelser
for at lære effektivt

Tommelfingerregel: 10/24/7

Bliv ved, indtil

- teksten føles velkendt og let forståelig
- du kan løse opgaverne uden hjælp

Brug din studiegruppe så meget som muligt

- brug de nye begreber mundtligt: spørg, forklar, eksemplificér
- vær konkret og specifik

ERHVERVSAKADEMI
AARHUS

Reaktioner på i dag

Spørgsmål?

Bekymringer?

Protester?

Kritik?